CSci 21 Data Structures and Algorithms
Module 6

Hash Tables

Learning Outcomes

LO 6.1 Implement accurate hash table operations

LO 6.2 Implement accurate collision resolution techniques

LO 6.3 Analyze time complexity under different load factors

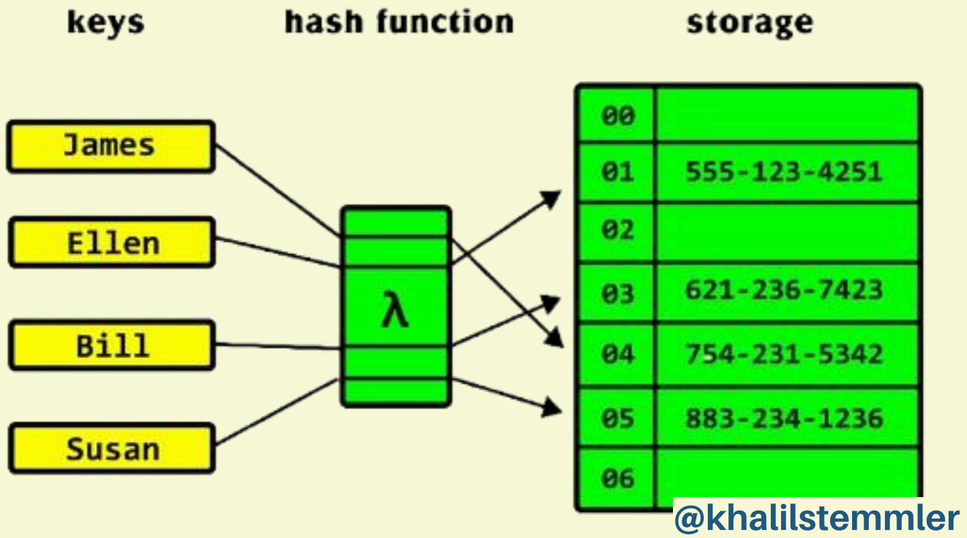
LO 6.4 Implement accurate solutions for problems using hash table ADT

Setting the Expectations

PREREQUISITES

- **Mastered** the **arrays, linked lists,** and **dynamic memory management concepts in C/C++**—allocation, deallocation, dereferencing, and redirection.
- Utilized **operator overloading** and **function pointers** in C/C++.

What's a Hash Table?

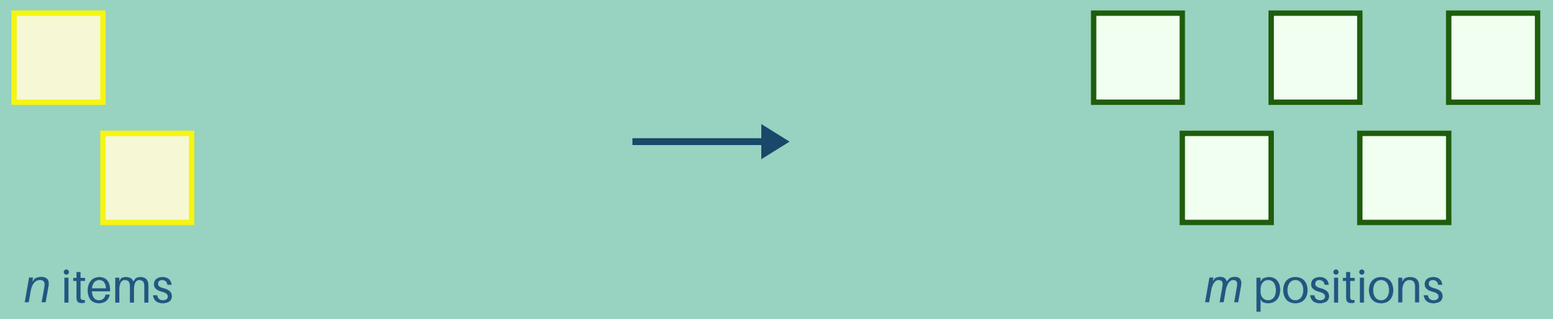


A data structure that maps keys to values using a hash function

Hash Table Basic Structure

```cpp
template<typename K, typename V>
class HashTable
{
private:
    struct HashNode {
        K key
        V value;
        HashNode* next;
        HashNode(K k, V v) : key(k), value(v), next(nullptr) {}
    };

    std::vector<HashNode*> table;
    int capacity;
    int size;

    int hashFunction(const K& key) const {
        return hash<K>{}(key) % capacity;
    }

public:
    HashTable(int cap) : capacity(cap), size(0)
    {
        table.resize(capacity, nullptr);
    }

    //main and auxiliary operations implementation
};
```

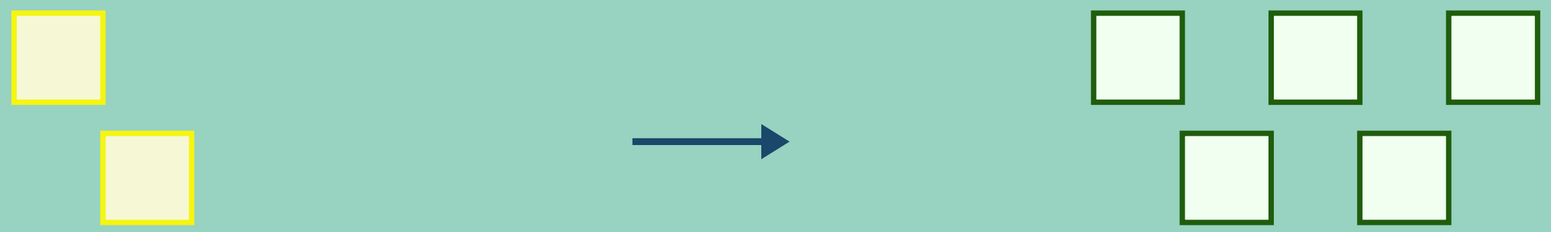
Hash Functions



m^n hash functions

$$\frac{m!}{(m-n)!}$$ **perfect hash functions**

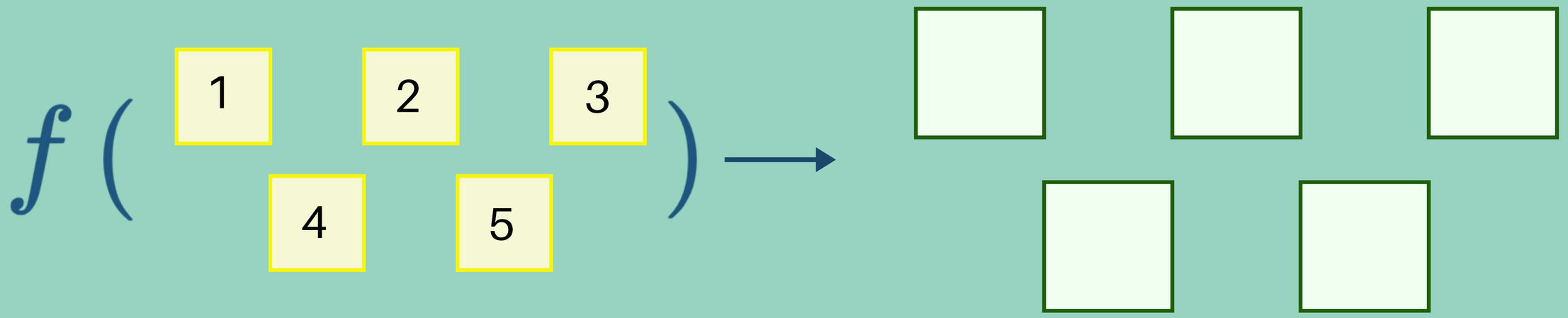
Hash Functions



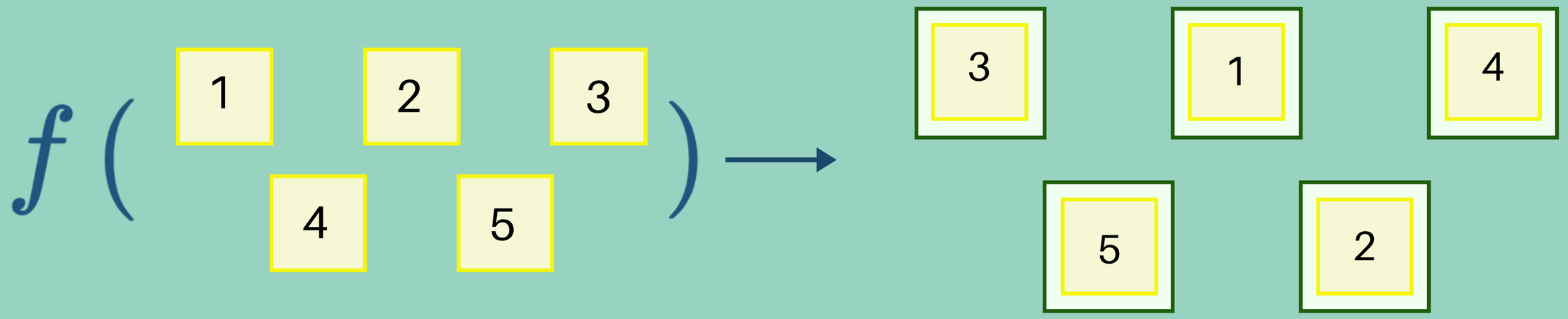
***5^2* or 25** hash functions

$$\frac{5!}{(5-2)!} = \frac{5 \times 4 \times 3!}{3!} = 20$$ **perfect hash functions**

Perfect Hash Functions

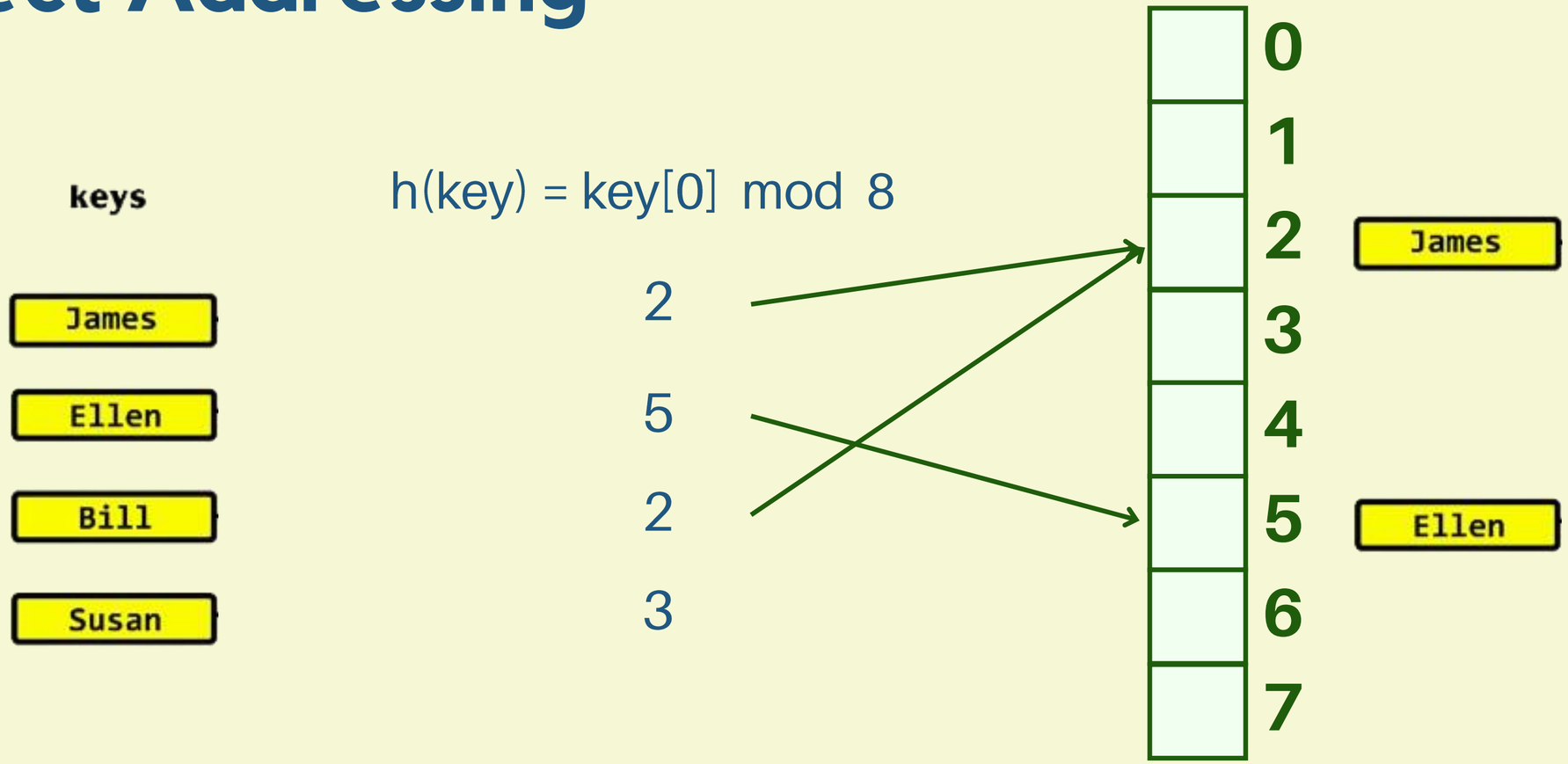


Perfect Hash Functions

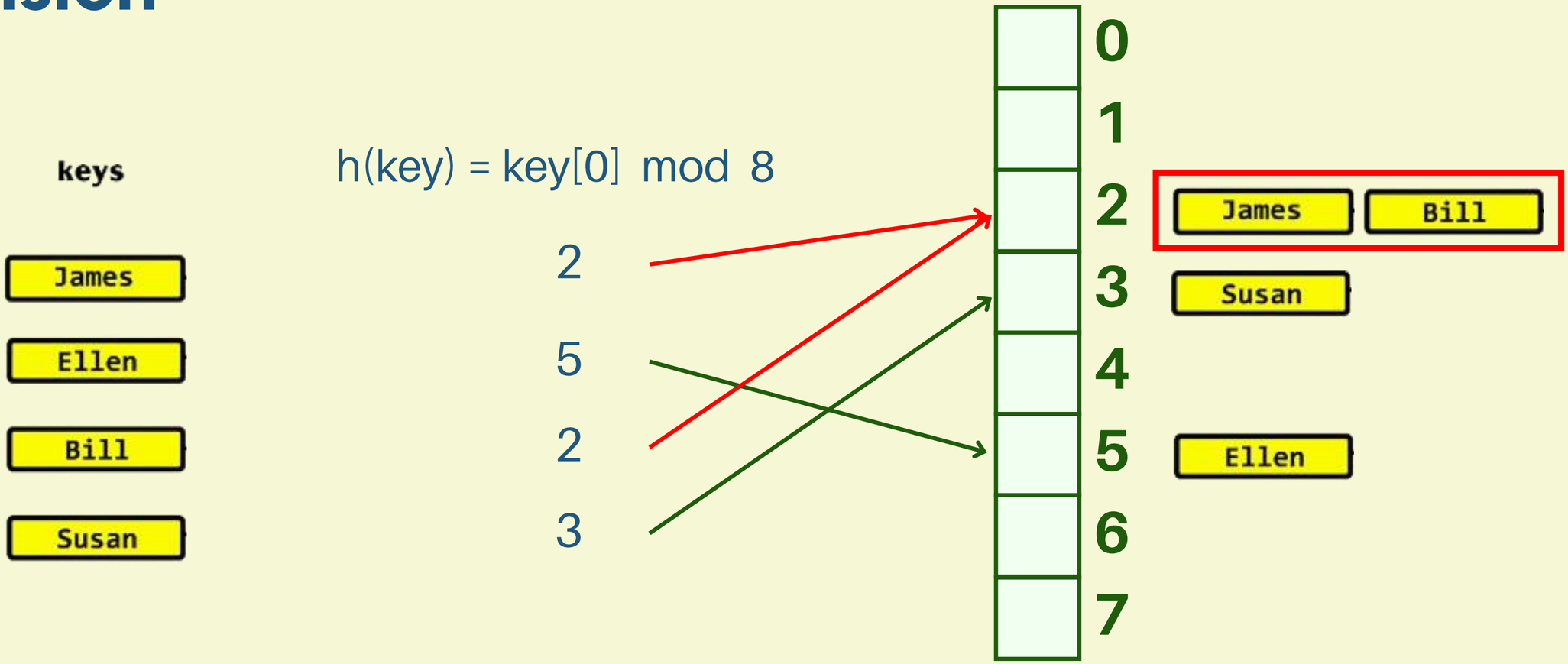


bijective

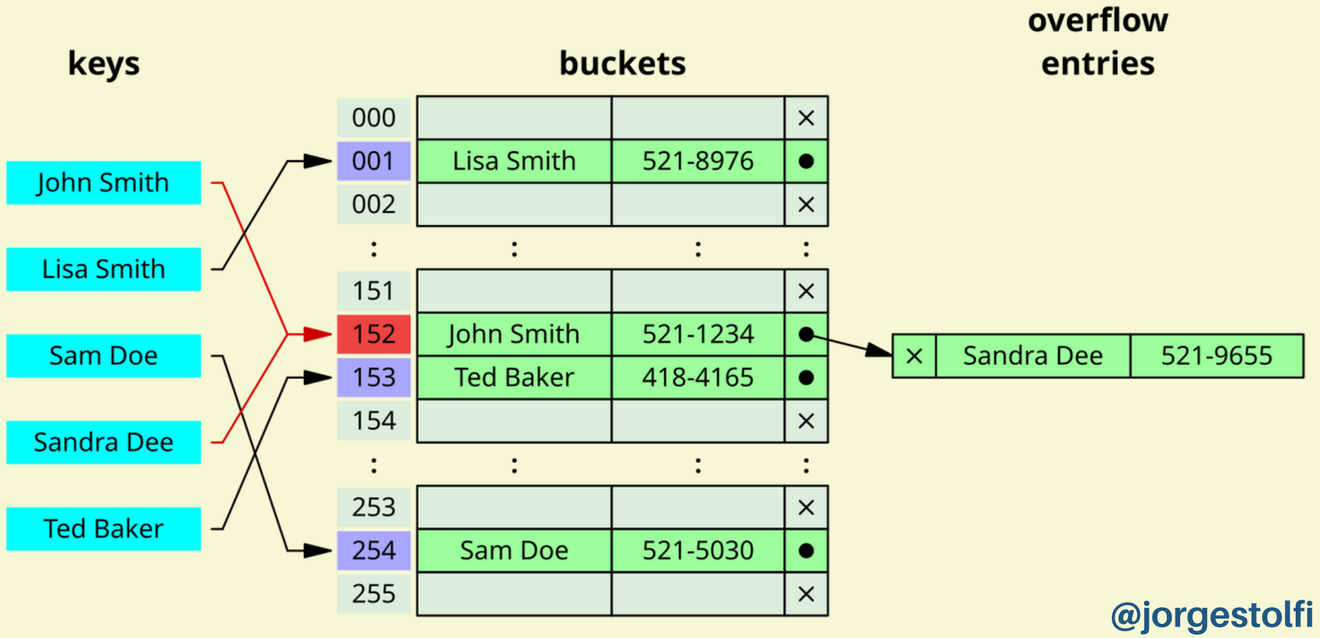
Direct Addressing



Collision


keys
James
Ellen
Bill
Susan
h(key) = key[0] mod 8
2
5
2
3
0
1
2
3
4
5
6
7
James
Bill
Susan
Ellen

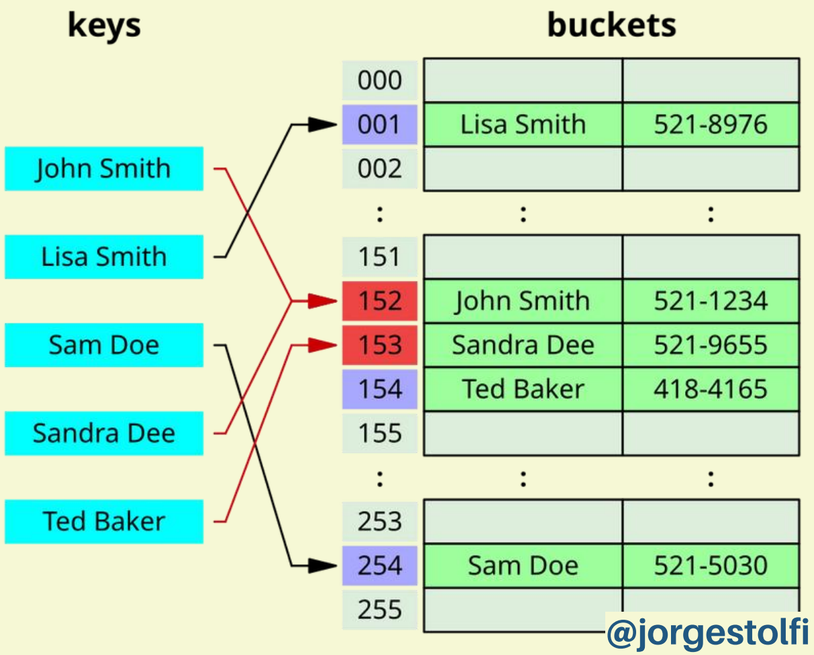
Collision Resolution



@jorgestolfi

separate chaining

Collision Resolution



open addressing

Separate Chaining

array of linked lists

Open Addressing: Linear Probing

$$h_p(x) = h(x) + i \mod m$$

hp(**x**) is the probing hash function
h(**x**) is the original hash function
i is the iteration 0, 1, 2, ...

Open Addressing: Quadratic Probing

$$h_p(x) = h(x) + i^2 \mod m$$

hp(**x**) is the probing hash function
h(**x**) is the original hash function
i is the iteration 0, 1, 2, ...

Open Addressing: Double Hashing

$$h_p(x) = h(x) + i \cdot h_2(x) \ mod \ m$$

hp(**x**) is the probing hash function
h(**x**) is the original hash function
h2(**x**) is the second hash function
i is the iteration 0, 1, 2, ...

Load Factor

$$\alpha = \frac{n}{m}$$

n is the number of elements
m is the table size

Load Factor

	linear probing	quadratic probing	double hashing
successful search	$\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$	$1-ln\left(1-\alpha\right)-\frac{\alpha}{2}$	$\frac{1}{\alpha}ln\frac{1}{1-\alpha}$
unsuccessful search	$\frac{1}{2}\left(1+\frac{1}{\left(1-\alpha\right)^2}\right)$	$\frac{1}{1-\alpha}-\alpha-ln\left(1-\alpha\right)$	$\frac{1}{1-\alpha}$

$\boldsymbol{\alpha}$ is the loading factor

Rehashing

1. Check the load factor
2. Create a new hash table
3. Rehash all keys
4. Transfer all key-value pairs to the new hash table
5. Dispose the old hash table

Complexity of Hash Table Operations

SCENARIO	TIME COMPLEXITY	WHEN?
Good/Perfect hash function	**O(1)**	**Uniform distribution**
Separate chaining	O(n)	Depends on chain length
Open addressing	O(n)	As α approaches 1

Key insight:

Load factor α is the dictator of hash table performance. Keep it below 0.7 for efficiency.

Application

Memoization

"The key to wisdom is constant and frequent questioning, for by doubting we are led to question, by questioning we arrive at the truth..."

- Peter Abelard

OPEN FORUM Q/A

for Module 6